Adding an Evo 2 score to a variant pipeline

A Nextflow pipeline that scores a case's SNVs with Evo 2 7B, and what it takes to run it reproducibly

Tim Richardson
Rewire

2 November 2026

Abstract

A score from a genome language model is only useful to a diagnostic lab if it runs inside a pipeline that gives the same answer each time and says what it did. We built a Nextflow pipeline that adds an Evo 2 7B zero-shot score to a case's single-nucleotide variants (SNVs), next to CADD v1.7 and SpliceAI, and ranks the case. A `gpu` profile scores 8,192 bp windows in a container pinned by digest with weights checked by SHA-256; a `replay` profile reads scores recorded on an H100 so the pipeline runs on a laptop. On a simulated BRCA1 case of 100 variants, the replay profile ran end to end in about 3 seconds, every variant received all three scores, and the diagnostic missense variant ranked 8th by Evo 2, 9th by CADD and 96th by SpliceAI (tied with 54 others at zero). Two runs gave identical ranked tables, a resumed run reused all six tasks, a mismatched reference allele was dropped and a variant with no recorded score stopped the run. Replaying scores from a second H100 gave identical output. Replaying scores from an L40S changed the Evo 2 rank of 80 of the 100 variants (median move 3 places, largest 30; the top five were unchanged and the diagnostic moved from 8th to 10th), although the scores differed by a median of only 1.0×10^{-4}. The recorded cloud runs that supplied the scores cost USD 2.82 to 4.78 each for 3,893 BRCA1 variants plus other workloads; scoring this case's 198 windows would take about 68 seconds of H100 time. The gpu profile has not been run end to end through Nextflow. The pipeline shows that the engineering is small; the open problem is that the score depends on the GPU type, so a lab must fix the GPU or record it with every score.

1 Introduction

A clinical lab that sequences a patient ends up with a list of candidate variants in the genes it tested. It ranks them with established scores, such as CADD[1] and SpliceAI[2], before a scientist reviews them. Genome language models such as Evo 2[3] score a variant zero-shot: the drop in the model's likelihood of the DNA sequence when the variant is put in. Unlike CADD and SpliceAI, Evo 2 has no genome-wide precomputed table, so a lab that wants the score has to run the model on a GPU.

This study asks what that involves in practice. It is an engineering study, not a test of accuracy. We wrote a Nextflow[4] pipeline that takes a case VCF, adds CADD, SpliceAI and Evo 2 7B scores, and ranks the case by each. We then checked four things on a simulated BRCA1 case: that it runs end to end; that it gives the same output when run again; that it fails safely on bad input; and how much the Evo 2 score, and so the ranking, depends on the GPU that computed it. Cost figures come from the recorded GPU runs whose scores the pipeline replays.

How well Evo 2 separates damaging from harmless BRCA1 variants was measured in a separate study[5], whose scores this pipeline reuses. Over 3,893 BRCA1 SNVs with saturation genome editing classes[6], Evo 2 7B had an AUROC of 0.875 (95% CI 0.859 to 0.891) and CADD v1.7

0.881 (0.862 to 0.897); the difference, -0.006 (-0.025 to +0.014), was not significant[5]. On missense variants CADD v1.7 was better (difference -0.042, -0.074 to -0.009)[5]. We do not repeat that comparison here.

2 Methods

The repository folder `evo2-pipeline/` holds the pipeline; `evo2-pipeline/paper/` holds the script that ran every check in this paper (`record_run.sh`), its outputs (`run/`), the chart script and this text. The pipeline code is unchanged since tag `amp-2026-v1`.

2.1 The pipeline

The pipeline is Nextflow DSL2 (`main.nf`) with seven processes, of which six run for a case (Figure 1). The profile chooses whether Evo 2 scores are computed or replayed.

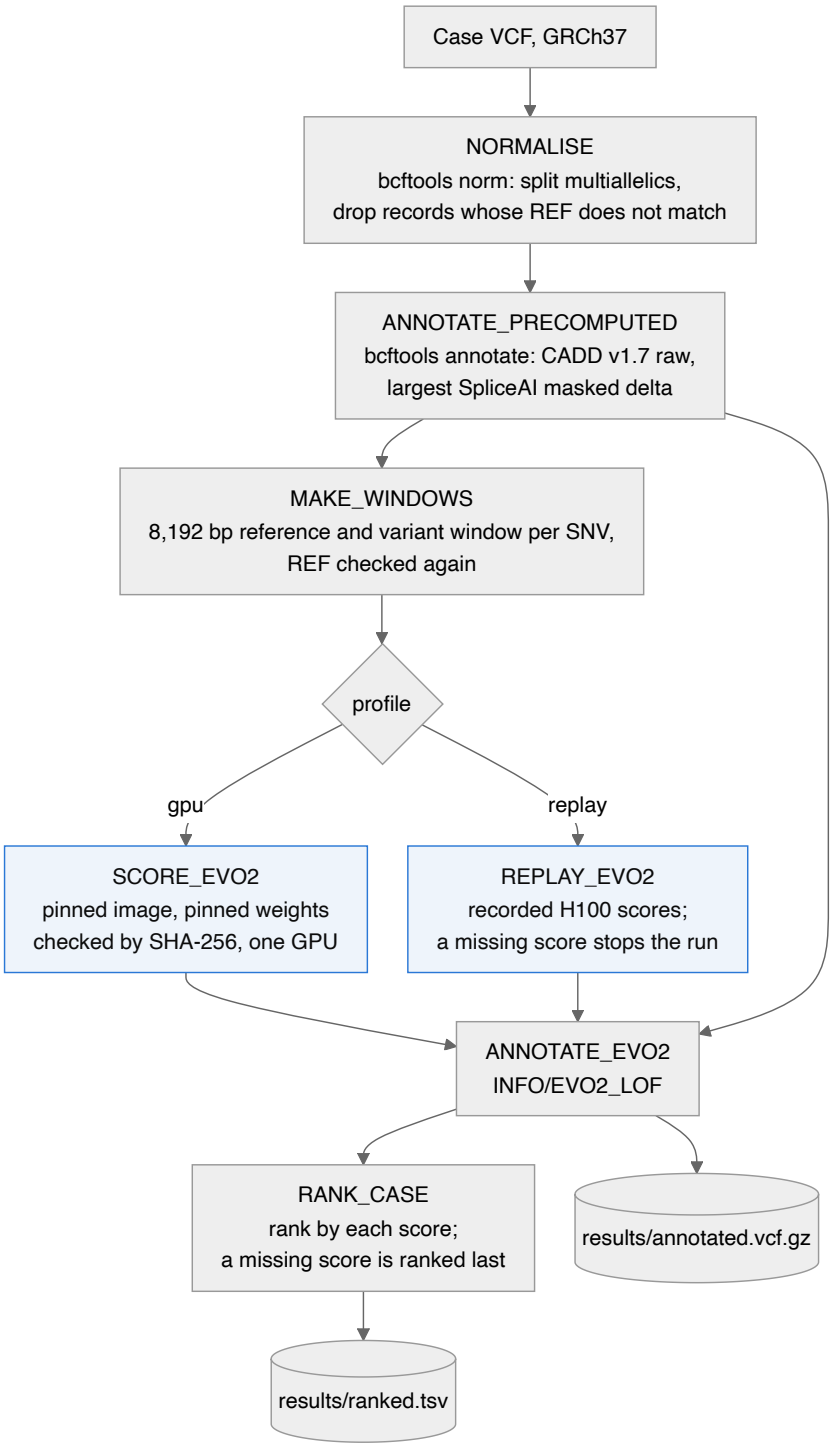


Figure 1. The pipeline. Six processes run for each case; the profile chooses SCORE_EVO2 (gpu) or REPLAY_EVO2 (replay). The two published outputs are at the bottom.

- **NORMALISE.** `bcftools norm --fasta-ref <ref> --multiallelics -any --check-ref x` splits multiallelic records and excludes any record whose REF allele does not match the reference[7,8].
- **ANNOTATE_PRECOMPUTED.** `bcftools annotate` adds two INFO fields from bgzipped, tabix-indexed tables: the CADD v1.7 raw score (`CADD_RAW`)[1] and `SPLICEAI_MAX`, the largest of the four masked SpliceAI delta scores[2], taken from the SpliceAI columns of CADD v1.7's GRCh37 annotation file, over BRCA1 rows only and ignoring NA. CADD

v1.7 uses SpliceAI as one of its features, so the two scores are not independent.

- **MAKE_WINDOWS.** For each SNV (`bcftools view --types snps`), `bin/make_windows.py` cuts an 8,192 bp reference window centred on the variant and the same window with the alternate base, as in Arc Institute's BRCA1 notebook[9]. It checks the reference base again and stops on a mismatch. Variants at one position share one reference window.
- **SCORE_EVO2** (gpu profile). `bin/score_evo2.py` downloads `arcinstitute/evo2_7b`[10] at revision `bda0089f92582d5baabf0f22d9fc85f3588f6b58`, checks the file's SHA-256 (`c66645929dc1...`) before `torch.load` unpickles it, and scores each window one at a time with `score_sequences` and `reduce_method="mean"`. Each reference window is scored once. The score `EVO2_LOF` is the reference window's mean log-likelihood per base minus the variant window's, so higher means more damaging. The process runs in the evo2-cloud-operations image pinned by digest (`sha256:22970720ff8b...`) with 64 GB of memory, a 3 hour limit and one task at a time (`nextflow.config`). It requests one GPU (`accelerator = 1`), but the container is started with `--gpus all`, so on a multi-GPU host it sees every GPU and Evo 2 uses the first.
- **REPLAY_EVO2** (replay profile). `bin/replay_evo2.py` looks each SNV up in a recorded score table and writes the same columns. A variant with no recorded score stops the run rather than becoming a missing value.
- **ANNOTATE_EVO2.** Adds `EVO2_LOF` to the VCF with a header line that states the model, the score and the window length, and publishes `results/annotated.vcf.gz`.
- **RANK_CASE.** `bin/rank_case.py` ranks the variants by each score separately, highest first, and publishes `results/ranked.tsv`. A missing score is written as `missing` and ranked last, never as zero: a missing CADD or SpliceAI value means the variant was not scored, not that it is benign. Ties keep the VCF order.

The two profiles differ only in the Evo 2 step. Only SCORE_EVO2 runs in a pinned container; the other steps use whatever bcftools, htslib and python3 are installed on the host. Figure 3 summarises what each control fixes and what it does not.

2.2 Resources and the test case

`scripts/prepare_resources.py` builds `resources/` from the evo2-cloud-operations study at commit `f8ae11c`[5]: GRCh37.p13 chromosome 17 renamed to `17`, bgzipped and indexed; the CADD v1.7 and SpliceAI extracts for the BRCA1 variants; and the recorded Evo 2 7B scores from that study's H100 run on Google Cloud (run `recorded-b75c67aa`, platform P2). `resources/` is not committed because the CADD and SpliceAI values may not be redistributed.

The same script builds the test case (Figure 2). From the 3,893 BRCA1 SNVs with saturation genome editing classes[6], it draws, with seed 20261016, one variant classed as loss of function (the diagnostic variant) and 99 variants not classed as loss of function at other positions. The case is written to `test/case_brca1.vcf` and the diagnostic variant to `test/case_brca1.truth.txt`. The case has 100 variants at 98 positions: 49 missense, 24 intronic, 14 splice region, 12 synonymous and 1 canonical splice. Of the 99 others, 94 are classed functional and 5 intermediate. The diagnostic variant is 17:41258546:A>C, a missense variant.

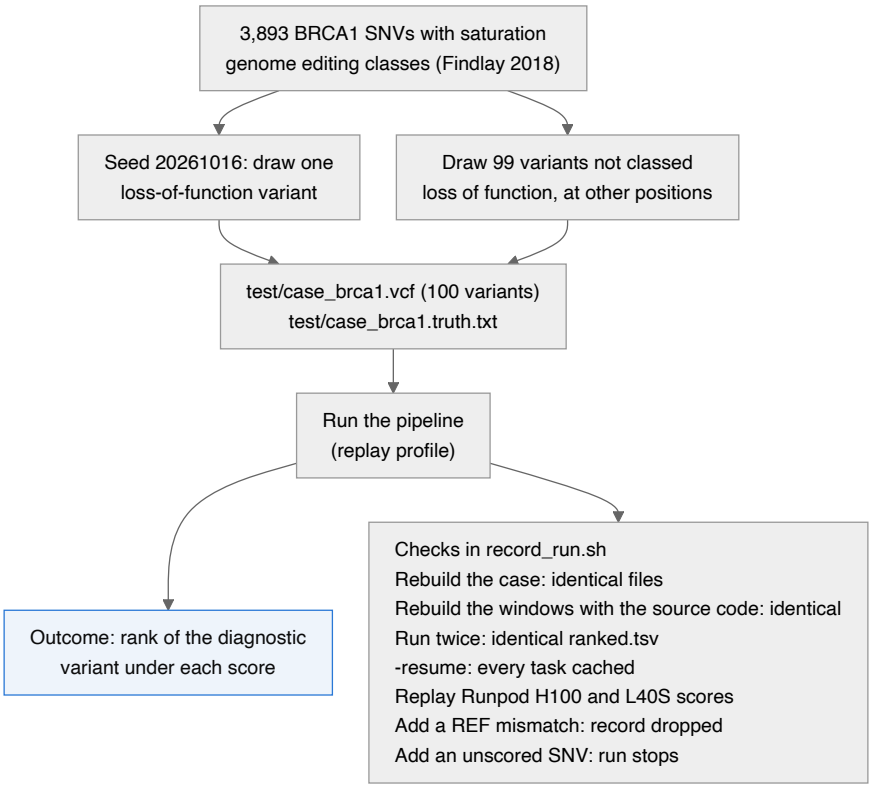


Figure 2. How the test case is built from the BRCA1 assay table, and the checks record_run.sh runs on it.

2.3 Checks

`paper/record_run.sh` copies the pipeline to a scratch directory, rebuilds `resources/` and `test/`, and runs the replay profile seven times, each in a fresh work directory unless stated:

1. `first` and `second`: the same case twice. Compare `ranked.tsv` byte for byte, and `annotated.vcf.gz` with and without the `##bcftools_*` header lines, which record the command and date.
2. `resume`: the second run again with `-resume`; count cached tasks.
3. `p3`: replay with the Runpod H100 scores for the same variants from the same recorded run (platform P3).
4. `l40s`: replay with the Runpod L40S scores (platform P5). Compare scores and Evo 2 ranks with the first run.
5. `mismatch`: the case plus one record at 17:41250000 (a BRCA1 intron, not in the assay) whose REF does not match the genome.
6. `missing`: the case plus one SNV at 17:41250000 with the correct REF and no recorded Evo 2 score.

It also checks that the rebuilt `test/` matches the committed files and that every replayed score equals the recorded score. Replay looks scores up by variant only, so it does not test the windows. After the first run, `record_run.sh` therefore rebuilds the case's windows with the source study's own code (`snv_windows` in evo2-cloud-operations `src/evo2ops/genome.py`, from its raw chromosome 17 FASTA) and compares them with the windows MAKE_WINDOWS built, by SHA-256. `summarise_run.py` writes `run/summary.json`, `run/ranks.tsv` (ranks, consequences and Evo 2 scores; no CADD or SpliceAI values) and the Nextflow trace of the first run. `make_charts.py` draws Figures 4 to 6 from those files.

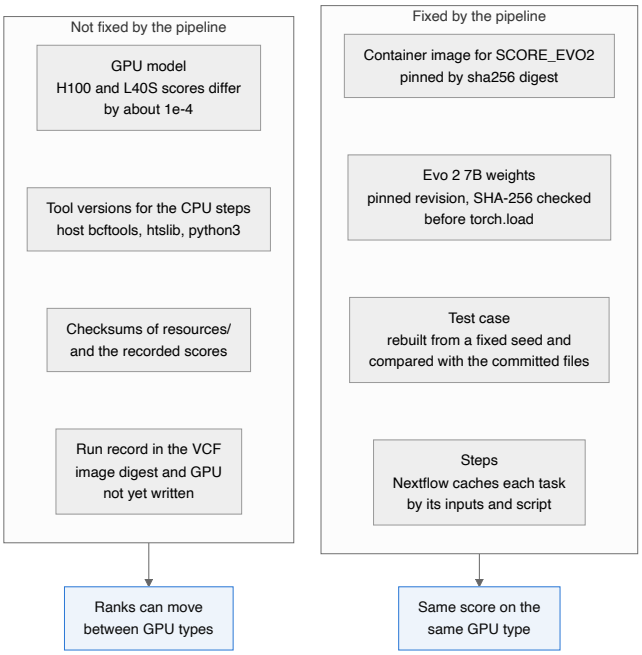


Figure 3. What the pipeline fixes, and what it does not. The GPU model and the run record in the VCF are the open items.

2.4 Cost and timing

GPU cost and timing come from the run records of evo2-cloud-operations run `recorded-b75c67aa` (`operations.json`), copied into `run/summary.json`[5]. Each platform ran the same workloads in the same image: the 3,893 BRCA1 variants (workload W1) and several smaller workloads. Cost is billed seconds times the instance price recorded at launch, plus storage. For the test case we estimate GPU time as the number of windows (one reference window per position plus one variant window per SNV) divided by the W1 rate on the replayed H100, and add the fixed start-up times recorded on Runpod. These estimates are arithmetic on recorded numbers, not measurements.

2.5 Software and statistics

The checks ran on macOS (Darwin 25.6.0, arm64) with Nextflow 24.10.5 (build 5935), OpenJDK 21.0.12.1, bcftools 1.24 with htslib 1.24, samtools 1.24 and Python 3.15.0. Charts used matplotlib 3.11.2 under Python 3.12. The recorded GPU runs used PyTorch 2.7.0a0 (NVIDIA build 25.04) with CUDA 12.9 and Transformer Engine in the pinned image[5].

This is one simulated case. Ranks are reported as observed, with the range of ranks a tied score could take. We make no test of significance on the case: a single case cannot show that one score ranks better than another. The pass or fail checks are deterministic. The L40S comparison was planned as a reproducibility check, but the statistics on it (median and largest difference, Spearman correlation, rank moves) are descriptive and exploratory. The AUROCs quoted in the Introduction are from evo2-cloud-operations, with 95% intervals from 2,000 resamples of positions[5].

3 Results

3.1 The pipeline runs end to end on a laptop

All seven replay runs except `missing` exited 0. The first run took 3.21 seconds of wall time; its six tasks took 46 ms (ANNOTATE_EVO2) to 720 ms (MAKE_WINDOWS) each (`run/trace_first.txt`). All 100 variants received a CADD, a SpliceAI and an Evo 2 score; none was ranked as missing. The rebuilt test case matched the committed files, and the 100 replayed Evo 2 scores equalled the recorded scores exactly (largest difference 0). The 198 windows MAKE_WINDOWS built were identical to those the source study's code builds for the same variants.

3.2 Where the scores rank the diagnostic variant

Table 1 and Figure 4 show the ranks. Evo 2 on the H100 scores put the diagnostic variant 8th of 100 (score 0.00343) and CADD v1.7 9th. SpliceAI gave it 0, as it did 54 other variants; the pipeline's rank, 96, is its place in VCF order among the tied variants, which could be anywhere from 46 to 100. The case's one canonical splice variant (17:41219623:A>T), classed intermediate by the assay, ranked 1st by CADD, 2nd by Evo 2 and 3rd by SpliceAI. None of these differences is tested; with one case none could be significant.

Table 1. Rank of the diagnostic variant 17:41258546:A>C among the case's 100 variants, and the number of the 100 variants each score covered. The L40S row is exploratory. Source: `run/summary.json`.

Score	Rank	Ranks a tie allows	Scored
CADD v1.7 raw	9	9	100
SpliceAI, largest masked delta	96	46 to 100 (55 variants at 0)	100
Evo 2 7B, H100 scores	8	8	100
Evo 2 7B, L40S scores	10	10	100

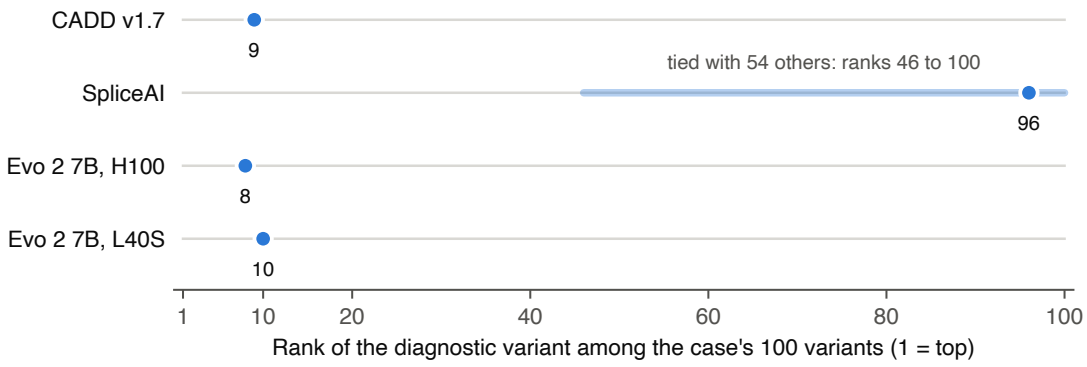


Figure 4. Rank of the diagnostic variant under each score, out of 100 (1 is the top). The shaded bar is the range a tied SpliceAI score could take.

3.3 The same input gives the same output

Two fresh runs gave byte-identical `ranked.tsv` files (Table 2). The two `annotated.vcf.gz` files differed only in the `##bcftools_*` header lines, which carry the run date; without those lines they were identical. Rerunning with `-resume` reused all six tasks from the cache and submitted none, in 1.43 seconds. Replaying the Runpod H100 scores gave a `ranked.tsv` identical to the Google Cloud H100 run: the two H100s gave the same score for every variant, as they did for

all 3,893 BRCA1 variants in the source study (largest difference 0)[5].

3.4 Bad input is caught

With a record whose REF did not match the genome, `bcftools norm` counted 101 records and removed 1 as a mismatch. The run finished and ranked the other 100. With an SNV that had no recorded Evo 2 score, REPLAY_EVO2 stopped with "No recorded Evo 2 score for chr17:41250000:A>C" and the run exited 1, so no ranked table was written with a gap in it.

Table 2. Pass or fail checks from `record_run.sh`. Source: `run/summary.json`.

Check	Result
Rebuilt test case matches the committed files	Yes
Pipeline windows equal the source study's windows	Yes, 198 of 198 (SHA-256)
Replayed scores equal the recorded H100 scores	Yes, largest difference 0
Two fresh runs give the same `ranked.tsv`	Yes, byte for byte
Two fresh runs give the same `annotated.vcf.gz`	Yes, apart from the dated `##bcftools_*` header lines
`-resume` reuses completed tasks	6 of 6 cached, 0 submitted
Second H100 (Runpod) gives the same `ranked.tsv`	Yes
REF mismatch	Record dropped (1 of 101), run continues
SNV with no recorded score	Run stops, exit 1

3.5 A different GPU changes the ranking

Replaying the L40S scores for the same 100 variants changed their scores by a median of 1.01×10^{-4} and at most 5.21×10^{-4} (Figure 5). The Spearman correlation between the H100 and L40S scores was 0.966. The Evo 2 rank of 80 of the 100 variants changed, by a median of 3 places: 45 moved 2 places or fewer, 37 moved 5 or more and 16 moved 10 or more. The largest move, 30 places, was an intronic variant at H100 rank 80. Ranks 1 to 5 did not change, and the top 10 held the same 10 variants on both GPUs, in a different order. The diagnostic variant's score fell from 0.00343 to 0.00315 and its rank from 8th to 10th. These numbers are exploratory, from one case. They agree with the source study's comparison over all 3,893 BRCA1 variants: median difference 1.17×10^{-4}, largest 1.31×10^{-3}, Spearman 0.983[5]. In the source study, a repeat of 200 variants on the same machine gave identical scores on each of the three GPUs[5], so the change comes from the GPU type, not from run-to-run noise.

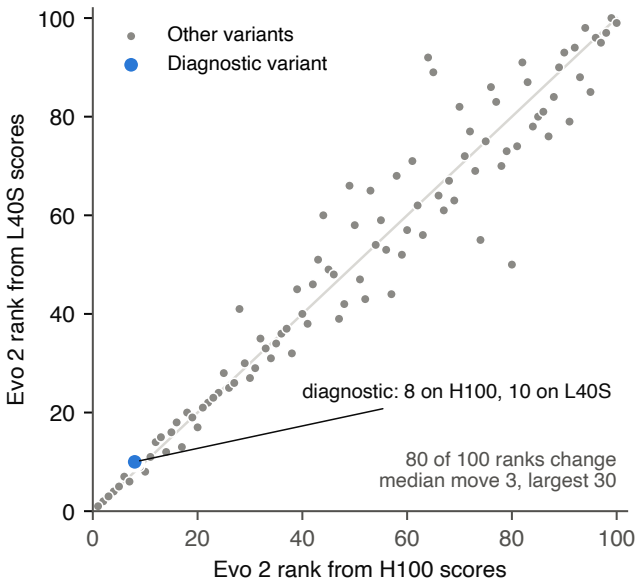


Figure 5. Evo 2 rank of each of the case's 100 variants from H100 scores (x) and from L40S scores (y). Points off the diagonal changed rank.

3.6 What the GPU step costs

Table 3 and Figure 6 give the recorded cloud runs. Each ran the 3,893 BRCA1 variants (5,219 windows: 1,326 reference and 3,893 variant) and other workloads. The Google Cloud H100, whose scores the replay profile uses, scored 2.93 windows a second and cost USD 4.78 for 2,734 billed seconds; its 491 second image pull fell inside the workload time. The Runpod H100 scored 2.91 windows a second and cost USD 2.82. The Runpod L40S was about five times slower on this host, at 0.61 windows a second, and its completed attempt cost USD 2.96. Speed depended on the host: in the source study, L40S pods in another Runpod data centre scored 0.92 to 0.93 windows a second, and in its follow-up an L40S scored 0.92, 3.2 times slower than the H100[5]. Peak GPU memory for the BRCA1 workload was 16.83 GiB on each H100 and 16.78 GiB on the L40S.

Table 3. The recorded GPU runs (evo2-cloud-operations run `recorded-b75c67aa`), as copied into `run/summary.json`. The Google Cloud Platform (GCP) H100 is the one the replay profile uses; the Runpod pods were on demand. Price is the instance price recorded at launch; storage added USD 0.0466 an hour on Google Cloud and USD 0.0274 on Runpod. Start is launch request to workload start; on Google Cloud the 491 s image pull falls after workload start and is not in Start, while on Runpod it is included. BRCA1 is the time for the 3,893 BRCA1 variants. Cost covers each completed attempt, including its other workloads. A first L40S attempt failed when the SSH session dropped and cost a further USD 0.82, so the recorded run cost USD 11.38 in all[5].

GPU, cloud	USD per hour	Start (s)	BRCA1 (s)	Windows/s	Billed (s)	Cost (USD)
H100, GCP spot	6.2426	74	1,782	2.93	2,734	4.78
H100, Runpod	3.99	401	1,796	2.91	2,526	2.82
L40S, Runpod	1.09	42	8,533	0.61	9,531	2.96

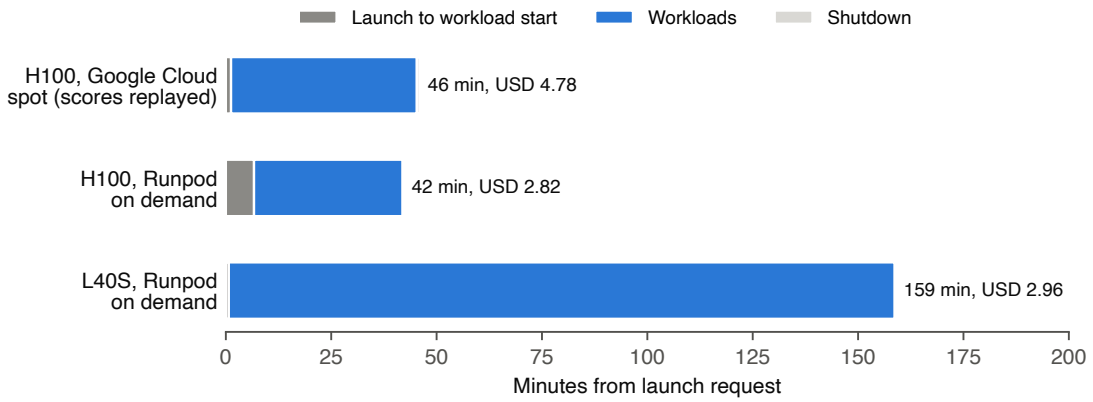


Figure 6. Time from launch request to shutdown for each recorded GPU run, with its cost. Shutdown took under a minute on each.

The test case needs 198 windows (98 reference, 100 variant). At 2.93 windows a second that is about 68 seconds of H100 time, or about USD 0.08 at Runpod's USD 3.99 an hour. Start-up dominates a job this small. On the Runpod H100 the pod took 401 seconds from launch to workload start, the weights took 50 seconds to download and check and 3.4 seconds to load, so one case scored on a fresh pod would take about 520 seconds, about USD 0.58. These are estimates from recorded rates, not measured runs of the pipeline.

4 Discussion

The engineering needed to add Evo 2 to a variant pipeline is small. Apart from the scoring step, the pipeline uses bcftools and two short Python scripts. The checks a lab would want before validation all passed on the replay profile: the output is the same when the input is the same, a resumed run reuses finished work, a reference mismatch is removed, and a missing score stops the run instead of turning into a silent gap. Keeping missing scores apart from zero matters; treating an unscored variant as benign would push it down the list for no reason.

The replay profile is useful in its own right. It lets the CPU parts of the pipeline be developed, tested and run in continuous integration on a laptop in seconds, against the exact scores the GPU produced. A lab could use the same idea to separate validating the pipeline from validating the model run.

The problem that remains is the GPU. Two H100s on different clouds gave identical scores, but L40S scores differed from them by a median of 1.0×10^{-4}, enough to move 80 of 100 variants in the ranking of one case, by a median of 3 places. NVIDIA guarantees identical cuBLAS results only on GPUs of the same architecture with the same number of multiprocessors[11]. In a follow-up on 487 BRCA1 variants, the source study found that full 32-bit precision brought H100 and L40S scores within a median of 4.8×10^{-7}, at 5.8 times the run time on the H100[5]. A lab therefore has two choices: fix the GPU type and validate on it, or pay for full precision. Either way the VCF should record which GPU, image digest and weights produced each score. The pipeline does not yet write the image digest or the GPU to the VCF header.

Cost is not the obstacle for a case-sized job. The model run for one case is about a minute of H100 time; most of the cost of a single-case job is starting the machine and loading the model. Batching many cases on one running GPU removes most of that overhead.

4.1 Limits

- **The gpu profile has not been run end to end through Nextflow.** `score_evo2.py` uses the same scoring call, image and weights as the run that produced the recorded scores, but its Nextflow wiring (Docker options, GPU request, cache mount) is untested.
- **One simulated case in one gene.** The case is built from assay variants, not from a patient, and every variant is in BRCA1 exons and nearby introns. The ranks say nothing general about which score is better.
- **Only the Evo 2 step runs in a pinned container.** The CPU steps use the host's bcftools, htslib and python3, and the pipeline does not checksum `resources/` or the recorded score table.
- **SNVs only.** MAKE_WINDOWS scores SNVs; indels pass through without an Evo 2 score and would be ranked last.
- **GRCh37 chromosome 17 only.** The resources cover the BRCA1 region of one chromosome.
- **Ties.** The pipeline breaks ties by VCF order. SpliceAI gives 0 to most variants, so its rank for a variant with a zero score is close to arbitrary; a lab should report the tie.
- **Costs are from one day's runs** at the prices recorded at launch, and the per-case figures are estimates.
- **The L40S comparison is from one case** and is exploratory.

5 Conclusion

A Nextflow pipeline can add an Evo 2 7B score to a case's SNVs with a few hundred lines of code, and its CPU steps can be run and tested on a laptop by replaying recorded GPU scores. On a simulated BRCA1 case it ran end to end, gave identical output on reruns and on a second H100, and failed safely on bad input. The score is reproducible on one GPU type but not across types: L40S scores moved 80 of 100 variants, by a median of 3 places, and the diagnostic variant from 8th to 10th. A lab adding Evo 2 should fix and record the GPU type, record the image and weights with each score, and run the gpu profile end to end before relying on it.

6 Data and code availability

Code: https://github.com/rewire-bio/amp-2026-studies, folder `evo2-pipeline/` (MIT licence). The pipeline code is unchanged since tag `amp-2026-v1`; this paper, `record_run.sh` and the run record in `paper/run/` were added after tag `amp-2026-v5`.

To reproduce, first fetch the inputs in evo2-cloud-operations. Its raw and derived data are not committed, so a fresh checkout is not enough. Check it out at commit `f8ae11c` next to this repository and run, in that checkout:

```
make inputs
uv run --frozen python followup/current_predictors.py --fetch
```

`make inputs` runs `scripts/data.py --fetch` and then `scripts/prepare_inputs.py`, which write the chromosome 17 FASTA and `data/derived/brca1_variants.tsv`. The second command queries the CADD v1.7 GRCh37 files remotely for the BRCA1 region and needs tabix, curl and unzip. Then, from the root of this repository:

```
bash evo2-pipeline/paper/record_run.sh ../evo2-cloud-operations
bash evo2-pipeline/paper/make_figures.sh
bash tools/paper/build.sh evo2-pipeline/paper
```

These need Nextflow 24.10 or later with Java 17 or later, bcftools, tabix, bgzip, samtools and python3 for the run; uv, mmdc and rsvg-convert for the figures; and pandoc 3 with XeLaTeX for the PDF. The versions used are under Software and statistics.

Inputs, from https://github.com/rewire-bio/evo2-cloud-operations at commit `f8ae11c`[5]: the recorded Evo 2 scores (committed there); GRCh37.p13 chromosome 17 from NCBI (public domain); the BRCA1 saturation genome editing table[6], free for non-profit use; CADD v1.7 scores[1], free for non-commercial use; and SpliceAI precomputed scores[2], Illumina's terms. CADD, SpliceAI and the assay table are not redistributed here; `prepare_resources.py` builds them locally. The Evo 2 7B weights[10] are Apache-2.0. GPU prices: Google Cloud Billing Catalog API (spot, us-central1) and the Runpod API at launch, as recorded in evo2-cloud-operations[5].

References

1. Schubach, M., Maass, T., Nazaretyan, L., Röner, S. & Kircher, M. CADD v1.7: Using protein language models, regulatory CNNs and other nucleotide-level scores to improve genome-wide variant predictions. *Nucleic Acids Research* **52**, D1143-D1154 (2024).
2. Jaganathan, K. *et al.* Predicting splicing from primary sequence with deep learning. *Cell* **176**, 535-548.e24 (2019).
3. Brixi, G., Durrant, M. G., Ku, J., *et al.* Genome modelling and design across all domains of life with Evo 2. *Nature* **652**, 1349-1361 (2026).
4. Di Tommaso, P. *et al.* Nextflow enables reproducible computational workflows. *Nature Biotechnology* **35**, 316-319 (2017).
5. Richardson, T. Evo 2 cloud operations: Scoring, cost and GPU reproducibility study. (2026).
6. Findlay, G. M. *et al.* Accurate classification of BRCA1 variants with saturation genome editing. *Nature* **562**, 217-222 (2018).
7. Danecek, P. *et al.* Twelve years of SAMtools and BCFtools. *GigaScience* **10**, giab008 (2021).
8. Danecek, P. *et al.* BCFtools manual page: Bcftools norm. (2026).
9. Arc Institute. ArcInstitute/evo2: Code, tests and notebooks (README, evo2/scoring.py, notebooks/brca1/brca1_zero_shot_vep.ipynb) at commit 53f1959. (2026).
10. Arc Institute. evo2_7b: Model card and weights. (2025).
11. NVIDIA. cuBLAS documentation, section 2.1.4: Results reproducibility. (2026).